

CS50P Week 2 Master Study Guide

Loops, Indexing & Collections

1. Core Mechanics: Iteration Protocols & Variable Values

A central concept in Week 2 is distinguishing what *i* holds during a loop versus what bracket resolutions like `value[i]` and `value[i][j]` evaluate to across different data structures.

What *i* Holds Across Loop Types

Context & Syntax	What <i>i</i> Evaluates To	Data Type of <i>i</i>	Example Output for <i>i</i>
List (Direct): <code>for i in fruits:</code>	Each element / value directly	String, Int, Dict, etc.	"apple", "banana"
List (Indexed): <code>for i in range(len(fruits)):</code>	The position number (index)	int (0, 1, 2, ...)	0, 1, 2
Dictionary (Direct): <code>for i in tea:</code>	Each key (string label)	Usually str	"green", "ginger"
Dictionary (.items()): <code>for key, value in tea.items():</code>	Unpacks key and value simultaneously	Tuple elements (str, Any)	key="green", value="mild"

Key Rule: A direct loop over a dictionary (`for i in dict:`) never yields values or numeric positions; it strictly iterates over the dictionary's *keys*.

2. Deep Dive: `container[i]` vs. `container[i][j]`

Brackets denote hierarchical depth:

- **One bracket (`container[i]`):** Steps 1 level down into the outer container.
- **Two brackets (`container[i][j]`):** Steps 2 levels down. Python resolves `container[i]` first, then applies `[j]` to that result.

Scenario A: Flat List vs. Indexed List

```
fruits = ["apple", "banana", "cherry"]

# Direct loop: i IS the element directly
for i in fruits:
    print(i) # apple, banana, cherry

# Index loop: i is the index, fruits[i] is the element
for i in range(len(fruits)):
```

Scenario B: List Inside a Dictionary

```
spacecraft = {
    "name": "James Webb",
    "instruments": ["NIRCam", "MIRI", "NIRSpec"]
}

# 1 Bracket: resolves to the entire sublist
sublist = spacecraft["instruments"] # ['NIRCam', 'MIRI', 'NIRSpec']

# 2 Brackets: [i] accesses dict key, [j] indexes the inner list
first_instrument = spacecraft["instruments"][0] # "NIRCam"
second_instrument = spacecraft["instruments"][1] # "MIRI"
```

Scenario C: Dictionary Inside a Dictionary (Nested Dict)

```
teastall = {
    "chai": {"masala": "spicy", "herbal": "grassy"},
    "tea": {"redtea": "red", "greentea": "mild"}
}

# 1 Bracket: extracts the inner dictionary
inner_dict = teastall["chai"] # {"masala": "spicy", "herbal": "grassy"}

# 2 Brackets: teastall["chai"]["masala"]
# Step 1: teastall["chai"] -> {"masala": "spicy", ...}
# Step 2: ["masala"] -> "spicy"
print(teastall["chai"]["masala"]) # spicy
```

Scenario D: List of Dictionaries (CS50 Students Pattern)

```
students = [
    {"name": "Hermione", "house": "Gryffindor", "patronus": "Otter"},
    {"name": "Harry", "house": "Gryffindor", "patronus": "Stag"}
]

# Indexed traversal:
# students[i] extracts the dictionary at index i
# students[i]["name"] accesses key inside that dictionary
for i in range(len(students)):
    print(students[i]["name"])

# Direct traversal:
# 'student' holds one dictionary at a time directly
for student in students:
    print(student["name"], student["house"], student["patronus"])
```

3. Comparison Matrix: Lists, Dictionaries & Tuples

Property	List (list)	Dictionary (dict)	Tuple (tuple)
Syntax	Square brackets: <code>[1, 2]</code>	Braces with pairs: <code>{"k": "v"}</code>	Parentheses: <code>(1)</code>

4. Built-in Methods Reference

Dictionary Methods

- `dict.get(key, default)` : Safely returns the value for `key` . If the key is absent, returns `default` (or `None`) without throwing a `KeyError` .
- `dict.items()` : Returns a dynamic view of (key, value) pairs as tuples, enabling two-variable unpacking (`for k, v in d.items():`). Calling `.items()` on a standard list triggers an `AttributeError` .
- `dict.keys()` : Returns a dynamic view of all keys (convertible to a list via `list(d.keys())`).
- `dict.values()` : Returns a dynamic view of all values.
- `dict.pop(key, [default])` : Removes the specified key and returns its value. Throws `KeyError` if missing and no default is provided.
- `dict.popitem()` : Removes and returns the last inserted (key, value) pair following LIFO order.
- `dict.clear()` : Empties all key-value entries in-place (`{}`).
- `in` keyword: `if key in dict:` checks existence across keys only in $O(1)$ average time.

List Methods & Cleaning Pipeline

- `.append(x)` : Adds `x` as a single element to the end of the list.
- `.extend(iterable)` : Unpacks an iterable and appends each element individually.
- `.insert(index, x)` : Inserts element `x` at position `index` , shifting subsequent items to the right.
- `.remove(val)` : Deletes the first occurrence of `val` (raises `ValueError` if absent).
- `.reverse()` : Reverses list elements in-place (mutates list directly, returns `None`).
- `separator.join(iterable)` : Concatenates an iterable of strings using `separator` as glue. Requires elements to be strings.
- `.strip().title()` : Cleans boundary whitespace and formats casing prior to list insertion.

5. Code Patterns & Control Flow Scripts

Sentinel Input Loop Pattern (`32_meow.py`)

```
def get_number():
    while True:
        n = int(input("what is the number of times you want the cat to meow : "))
        if n > 0:
            return n # Breaks loop and returns validated input simultaneously
```

String Repetition Overload (`30_multi.py`)

```
# Leverages Python's * operator overload for strings
print("MEOW!\n" * 3, end="")
```

Manual Numbering Tracker (`dictionary.py`)

```
count = 0
for key, val in tea.items():
    count += 1
    print(f"{count}. {key}: {val}")
```

6. Debugging Traps & Pitfalls

Pitfall 1: Plural vs. Singular Name Trap

```
# INCORRECT: Looking up key on the master list
for student in students:
    print(students["name"]) # TypeError: list indices must be integers

# CORRECT: Target the singular dictionary instance
for student in students:
    print(student["name"])
```

Pitfall 2: Scope Leak in Sequential Loops (`dict_in_dict.py`)

Loop variables in Python leak into the surrounding scope and remain bound to the last processed value:

```
for i, j in teastall.items():
    print(i, j)

# 'j' remains in memory bound ONLY to the final item ('tea')
for item, flavor in j.items():
    print(item, flavor) # Operates only on 'tea', completely skipping 'chai'

# FIX: Proper nested indentation inside the first loop
for i, j in teastall.items():
    for item, flavor in j.items():
        print(item, flavor)
```

Pitfall 3: Unbound Local Accumulators

Reusing or calling `count += 1` inside a loop without defining `count = 0` immediately prior triggers an `UnboundLocalError`. Counters must be explicitly initialized in the outer scope.

7. Week 2 Problem Set Post-Mortem

1. Camel Case (`camel.py`)

- **Logic:** Iterate through each character `char` in user input.
- **Rule:** If `char.isupper()`, append an underscore `_` followed by `char.lower()`; otherwise, append `char` unchanged.
- **Takeaway:** Strings are immutable; accumulate results into a string or a list formatted via `"".join()`.

2. Coke Machine (`coke.py`)

- **Logic:** Maintain a balance using `while amount_due > 0:`.
- **Validation:** Only decrement if the inserted coin is in `[25, 10, 5]`.
- **Change Computation:** When the loop finishes (balance reaches 0 or negative), compute change owed using `abs(amount_due)`.

3. Just setting up my twttr (`twttr.py`)

- **Logic:** Traverse string character by character and omit vowels.
- **Check:** `if char.lower() not in ["a", "e", "i", "o", "u"]:` (or `not in "aeiou"`).
- **Takeaway:** Check membership using `.lower()` while appending the unaltered `char` to maintain original consonant capitalization.

4. Vanity Plates (`plates.py`)

Validation Order:

- Length check: `2 <= len(s) <= 6`
- First two characters alphabetic: `s[0:2].isalpha()`
- Alphanumeric character set: `s.isalnum()`
- **Number Logic:** Locate the first digit. Verify it is not `'0'`, and confirm all subsequent characters to the end (`s[i:]`) are numeric (`s[i:].isdigit()`).

5. Nutrition Facts (`nutrition.py`)

- **Logic:** Store 20 FDA fruit items and caloric values in a dictionary with lowercase keys.
- **Sanitization:** Normalize user input using `.lower()`.
- **Safe Retrieval:** Query with `if item in fruits:` before printing `Calories: {fruits[item]}` to prevent `KeyError` crashes on invalid inputs.